

- **Browser caching:** As Web developers might be aware, some data can be cached on the client-side in the browser, like images, etc, so that they are automatically used when repeated requests for that resource are made.
- **Content delivery network:** A content delivery network is a network of servers that are geographically dispersed; users closest to a particular server in the CDN are then served by that server, reducing data transfer time from far-off servers, and also taking the load off the primary servers.
- **Reverse caching proxies:** A reverse caching proxy is another architectural option, where a proxy server sits between the client and the server. Here the client requests resources from the proxy server, which in turn either retrieves the resources on the client's behalf, or returns the data present in its cache. So, the client feels as if the proxy server is the source of information.
- **Server-side caching:** Data or objects can alternatively be cached on the server-side itself. This can either be a local server cache, a centralised caching server or a distributed cache.
- **Local database query cache:** A good database caches the database queries or data internally, so as to improve the speed of looking up data as well as the performance of the database.

You may choose to implement a cache in one way or another, or you might use a combination of more than one technique to cache different types of data at separate levels. But more importantly, it is helpful to know whether you even need caching in the particular application/use-case you are thinking about. Most people, in the process of implementing a cache, actually lose out because it was wrongly implemented. So the cache ends up slowing down the application, instead of speeding it up. Getting fancy software with fancy features doesn't always make sense, but using even the modest ones in the right way, does.

Why you need Memcached

This discussion assumes that you have set up a cluster, and you want to implement caching. In this case, what happens if you start caching on each node independently? You will see that some nodes face memory issues, while others have quite a bit of memory left. Moreover, most of the data stored in their individual caches is redundant. This calls for a centralised caching mechanism that makes sense in the cloud domain, such that the data being cached in a cluster is evenly distributed and unique for the whole cluster. And yes, the answer is Memcached.

Memcached is that piece of the puzzle without which your cloud implementation does not even make sense. It provides a solution in which the available memory in the cache is the sum of that on all nodes on which the Memcached instance is running. So if, for example,

you have 10 nodes, with each being allocated 1 GB of memory for caching, you get a total of 10 GB of cache available for the whole cluster. Here are some features in Memcached that might lure you into using it within the context of your application:

- **Easy scalability:** This feature is applicable for almost any software with the tag of 'distributed', but still, it is worth noting that Memcached needs minimal configuration to add a new node, with almost no special interconnect requirements. Your available memory in the cache just increases on the fly.
- **Hidden complexity:** Memcached hides beneath it all the complexity associated with storing/retrieving the data from the cache in a cluster. All we need to provide is the key associated with the data. The whole task of determining which node to store the data on, or to retrieve it from, is done by the Memcached client itself.
- **Minimal impact of a node failure:** Even if a particular Memcached node does fail, it has almost no impact on the overall cache other than reducing the available memory, and a minor increase in the number of cache misses.
- **Flexible architecture:** Memcached does not impose a restriction on all nodes to have a uniform cache size. So, some of your nodes with less physical memory can be set up to contribute perhaps only 512 MB to the cluster, while others may have 2 GB of memory dedicated for the Memcached instance. Apart from this, you can even run more than one instance of Memcached on a single node.
- **Multiple clients available:** Memcached has client APIs available for various languages like PHP, C++, Java, Python, Ruby, Perl, .NET, Erlang, ColdFusion and even more.
- **Cross-platform:** Memcached is available for a wide variety of platforms including Linux, BSD and Windows.
- **Multi-fetch:** With the help of this feature, we can request values for more than one key at once, instead of querying them in a loop, one by one, which takes a lot of network round-trips.
- **Constant time functions:** It takes the same amount of time to perform an operation in memory, whether it is a single key or a hundred. This corresponds to the Multi-fetch feature discussed before.

There are many more features that I may have missed, but I am just skimming the surface at the architectural level.

How it works

Memcached works on some very strong and unique principles, which you, as a developer, should know about in order to develop effective applications with it. Listed below are some of the major principles behind how Memcached handles data under the hood: